

FPGA ASSESSMENT 5

Anonymous public copy

Personal and institutional identifiers removed.

Contents

1	Aim	2
2	Architecture Diagram	2
3	Control Unit State Diagram	3
4	Working Principle	3
5	Verilog Code	4
5.1	Top Module	4
5.2	Control Unit	4
5.3	Datapath and Registers	6
5.4	Seven-Segment and Decimal Conversion Modules	7
6	Testbench and Verification	7
7	ModelSim Simulation Output	8
8	FPGA Implementation Output	8
9	FPGA Project and Pin Assignment	10
10	Conclusion	11

1. Aim

To design and implement a 4-bit serial multiplier in Verilog HDL using a control unit and datapath, and to display the decimal product on the DE2-115 seven-segment displays. The design is verified through simulation and FPGA implementation.

2. Architecture Diagram

The architecture contains only the control unit and datapath elements required for the multiplication algorithm. The seven-segment decoder is intentionally kept outside the architecture diagram.

FIGURE 18-7
Block Diagram for
Binary Multiplier
© Cengage Learning 2014

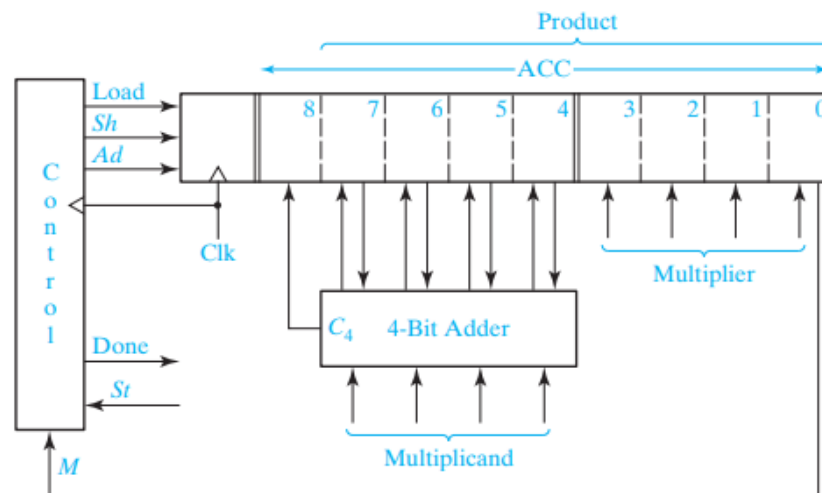


Figure 1: Architecture of the serial multiplier control unit and datapath.

The 9-bit accumulator stores the combined partial-product word. During a load operation, the multiplier occupies the least-significant four bits. In each ADD/SKIP state, the controller observes $m = q[0]$ and enables the 4-bit adder only when that bit is one. The following SHIFT state performs a right shift of the complete 9-bit word with zero fill. After four iterations, the controller asserts **done** and the accumulator contains the product.

3. Control Unit State Diagram

FIGURE 18-8
State Graph for
Multiplier Control
 © Cengage Learning 2014

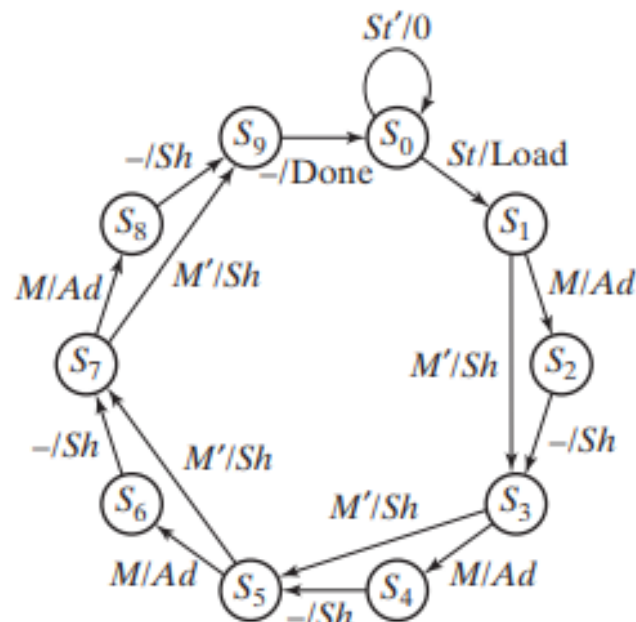


Figure 2: Finite-state-machine sequence implemented by the control unit.

State	Function	Output control
S0	Idle / wait for start	load=0, ad=0, sh=0, done=0
S1	Load multiplier into accumulator	load=1
S2	ADD/SKIP iteration 0	ad=m
S3	Shift iteration 0	sh=1
S4	ADD/SKIP iteration 1	ad=m
S5	Shift iteration 1	sh=1
S6	ADD/SKIP iteration 2	ad=m
S7	Shift iteration 2	sh=1
S8	ADD/SKIP iteration 3	ad=m
S9	Shift iteration 3	sh=1
S10	Completed	done=1

4. Working Principle

1. Reset places the FSM in S0 and clears the accumulator.
2. A start pulse moves the FSM from S0 to S1, where the multiplier is loaded into the lower four bits of the accumulator.
3. In every ADD/SKIP state, $q[0]$ is examined. If it is one, the multiplicand is added to $q[7:4]$ and the carry is stored in $q[8]$.
4. The next SHIFT state performs $q \leftarrow \{1'b0, q[8:1]\}$. This moves the carry/partial-product information toward the multiplier field while filling the new MSB with zero.
5. Four ADD/SHIFT iterations are sufficient for two 4-bit operands. The FSM then enters S10 and asserts **done** for one clock interval.
6. The final 9-bit accumulator value is converted to decimal digits and applied to HEX2, HEX1 and HEX0.

5. Verilog Code

5.1 Top Module

```

1 module serial_mul(clk,rst,st,multiplicand,multiplier,done,HEX0,HEX1,HEX2);
2
3 input clk,rst,st;
4
5 input [3:0]multiplicand;
6 input [3:0]multiplier;
7
8 output done;
9 output [6:0]HEX0,HEX1,HEX2;
10
11 wire load,sh,ad,m;
12 wire [8:0]product;
13 wire [3:0]hundreds,tens,ones;
14
15 control_unit c1(
16     .clk(clk),
17     .rst(rst),
18     .st(st),
19     .load(load),
20     .sh(sh),
21     .ad(ad),
22     .done(done),
23     .m(m)
24 );
25
26 datapath d1(
27     .clk(clk),
28     .rst(rst),
29     .load(load),
30     .sh(sh),
31     .ad(ad),
32     .multiplicand(multiplicand),
33     .multiplier(multiplier),
34     .product(product),
35     .m(m)
36 );
37
38 bin_to_bcd b1(
39     .product(product),
40     .hundreds(hundreds),
41     .tens(tens),
42     .ones(ones)
43 );
44
45 seven_segment s0(
46     .in(ones),
47     .hex(HEX0)
48 );
49
50 seven_segment s1(
51     .in(tens),
52     .hex(HEX1)
53 );
54
55 seven_segment s2(
56     .in(hundreds),
57     .hex(HEX2)
58 );
59
60 endmodule

```

5.2 Control Unit

```

1 module control_unit(clk,rst,st,load,sh,ad,done,m);
2
3 input clk,rst,st,m;
4
5 output reg load,sh,ad,done;
6
7 reg [3:0]cs,ns;

```

```

8
9 parameter s0  =4'b0000;
10 parameter s1  =4'b0001;
11 parameter s2  =4'b0010;
12 parameter s3  =4'b0011;
13 parameter s4  =4'b0100;
14 parameter s5  =4'b0101;
15 parameter s6  =4'b0110;
16 parameter s7  =4'b0111;
17 parameter s8  =4'b1000;
18 parameter s9  =4'b1001;
19 parameter s10 =4'b1010;
20
21 //cs logic
22 always@(posedge clk or posedge rst)
23 begin
24     if(rst)
25         cs<=s0;
26     else
27         cs<=ns;
28 end
29
30 //ns logic
31 always@(*)
32 begin
33     case(cs)
34         s0:
35             begin
36                 if(st)
37                     ns=s1;
38                 else
39                     ns=s0;
40             end
41         s1: ns=s2;
42         s2: ns=s3;
43         s3: ns=s4;
44         s4: ns=s5;
45         s5: ns=s6;
46         s6: ns=s7;
47         s7: ns=s8;
48         s8: ns=s9;
49         s9: ns=s10;
50         s10: ns=s0;
51         default: ns=s0;
52     endcase
53 end
54
55 //output logic
56 always@(*)
57 begin
58     load=1'b0;
59     sh=1'b0;
60     ad=1'b0;
61     done=1'b0;
62
63     case(cs)
64         s1: begin
65             load=1'b1;
66         end
67         s2: begin
68             ad=m;
69         end
70         s3: begin
71             sh=1'b1;
72         end
73         s4: begin
74             ad=m;
75         end
76         s5: begin
77             sh=1'b1;
78         end
79         s6: begin
80             ad=m;
81         end
82         s7: begin
83             sh=1'b1;
84         end
85         s8: begin

```

```

86         ad=m;
87     end
88     s9: begin
89         sh=1'b1;
90     end
91     s10: begin
92         done=1'b1;
93     end
94     default: begin
95     end
96 endcase
97 end
98
99 endmodule

```

5.3 Datapath and Registers

```

1 module datapath(clk,rst,load,sh,ad,multiplicand,multiplier,product,m);
2
3 input clk,rst,load,sh,ad;
4
5 input [3:0]multiplicand;
6 input [3:0]multiplier;
7
8 output [8:0]product;
9 output m;
10
11 wire [8:0]acc;
12 wire [3:0]sum;
13 wire cout;
14
15 assign m=acc[0];
16 assign product=acc;
17
18 accumulator a1(
19     .clk(clk),
20     .rst(rst),
21     .load(load),
22     .sh(sh),
23     .ad(ad),
24     .sum(sum),
25     .cout(cout),
26     .d(multiplier),
27     .q(acc)
28 );
29
30 adder4 a2(
31     .a(acc[7:4]),
32     .b(multiplicand),
33     .cin(1'b0),
34     .s(sum),
35     .cout(cout)
36 );
37
38 endmodule

```

```

1 module accumulator(clk, rst, load, sh, ad, sum, cout, d, q);
2     input clk, rst, load, sh, ad;
3     input [3:0] sum;
4     input cout;
5     input [3:0] d;
6     output reg [8:0] q;
7
8     always @(posedge clk or posedge rst) begin
9         if (rst)
10             q <= 9'b000000000;
11         else if (load)
12             q <= {5'b000000, d};
13         else if (ad) begin
14             q[7:4] <= sum;
15             q[8] <= cout;
16         end
17         else if (sh)
18             // Right-shift the 9-bit {carry, A, Q} word with zero fill.
19             q <= {1'b0, q[8:1]};

```

```

20     end
21 endmodule

```

```

1 module adder4(a,b,cin,s,cout);
2
3 input [3:0]a,b;
4 input cin;
5
6 output [3:0]s;
7 output cout;
8
9 assign {cout,s}=a+b+cin;
10
11 endmodule

```

5.4 Seven-Segment and Decimal Conversion Modules

```

1 module bin_to_bcd(product,hundreds,tens,ones);
2
3 input [8:0]product;
4
5 output reg [3:0]hundreds;
6 output reg [3:0]tens;
7 output reg [3:0]ones;
8
9 integer value;
10
11 always@(*)
12 begin
13     value=product;
14     hundreds=value/100;
15     tens=(value%100)/10;
16     ones=value%10;
17 end
18
19 endmodule

```

```

1 module seven_segment(in,hex);
2
3 input [3:0]in;
4 output reg [6:0]hex;
5
6 // DE2-115 seven-segment displays are common-anode.
7 // Segment order: {g,f,e,d,c,b,a}; 0 = ON, 1 = OFF.
8 always@(*)
9 begin
10     case(in)
11         4'd0: hex=7'b1000000;
12         4'd1: hex=7'b1111001;
13         4'd2: hex=7'b0100100;
14         4'd3: hex=7'b0110000;
15         4'd4: hex=7'b0011001;
16         4'd5: hex=7'b0010010;
17         4'd6: hex=7'b0000010;
18         4'd7: hex=7'b1111000;
19         4'd8: hex=7'b0000000;
20         4'd9: hex=7'b0010000;
21         default: hex=7'b1111111;
22     endcase
23 end
24
25 endmodule

```

6. Testbench and Verification

```

1 module serial_mul_tb();
2
3 reg clk,rst,st;
4 reg [3:0]multiplicand;
5 reg [3:0]multiplier;
6

```

```

7 wire done;
8 wire [6:0] HEX0, HEX1, HEX2;
9
10 serial_mul m1(
11     .clk(clk),
12     .rst(rst),
13     .st(st),
14     .multiplicand(multiplicand),
15     .multiplier(multiplier),
16     .done(done),
17     .HEX0(HEX0),
18     .HEX1(HEX1),
19     .HEX2(HEX2)
20 );
21
22 // Internal product is kept in the top module for simulation checking.
23 wire [8:0] product;
24 assign product = m1.product;
25
26 always
27     #5 clk=~clk;
28
29 initial
30 begin
31     clk=0;
32     rst=1;
33     st=0;
34     multiplicand=4'b0000;
35     multiplier=4'b0000;
36
37     #10 rst=0;
38
39     #10 multiplicand=4'b1010;
40     multiplier=4'b1011;
41     st=1;
42
43     #10 st=0;
44
45     #100 $stop;
46 end
47
48 endmodule

```

7. ModelSim Simulation Output

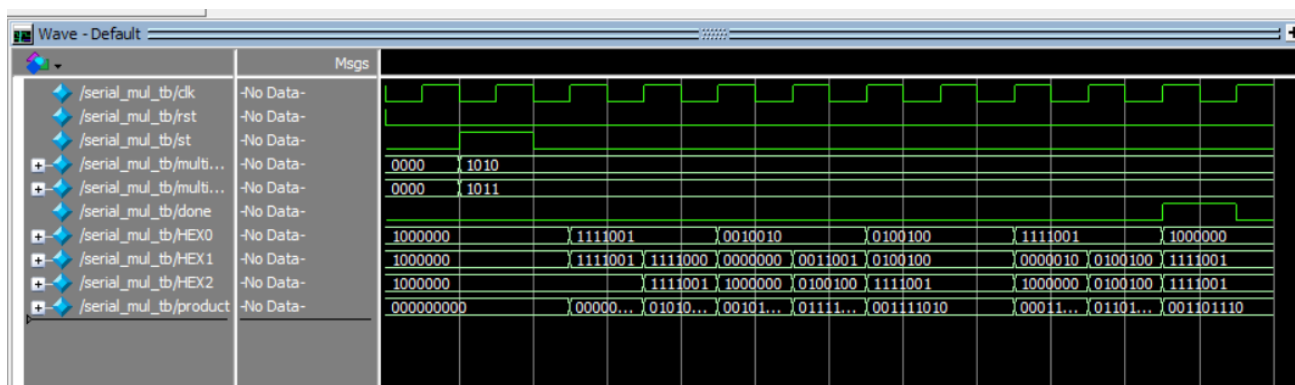


Figure 3: ModelSim simulation output of the `serial_mul_tb` testbench.

8. FPGA Implementation Output

The following photographs show the DE2-115 FPGA implementation results. The seven-segment display is visible in each image.

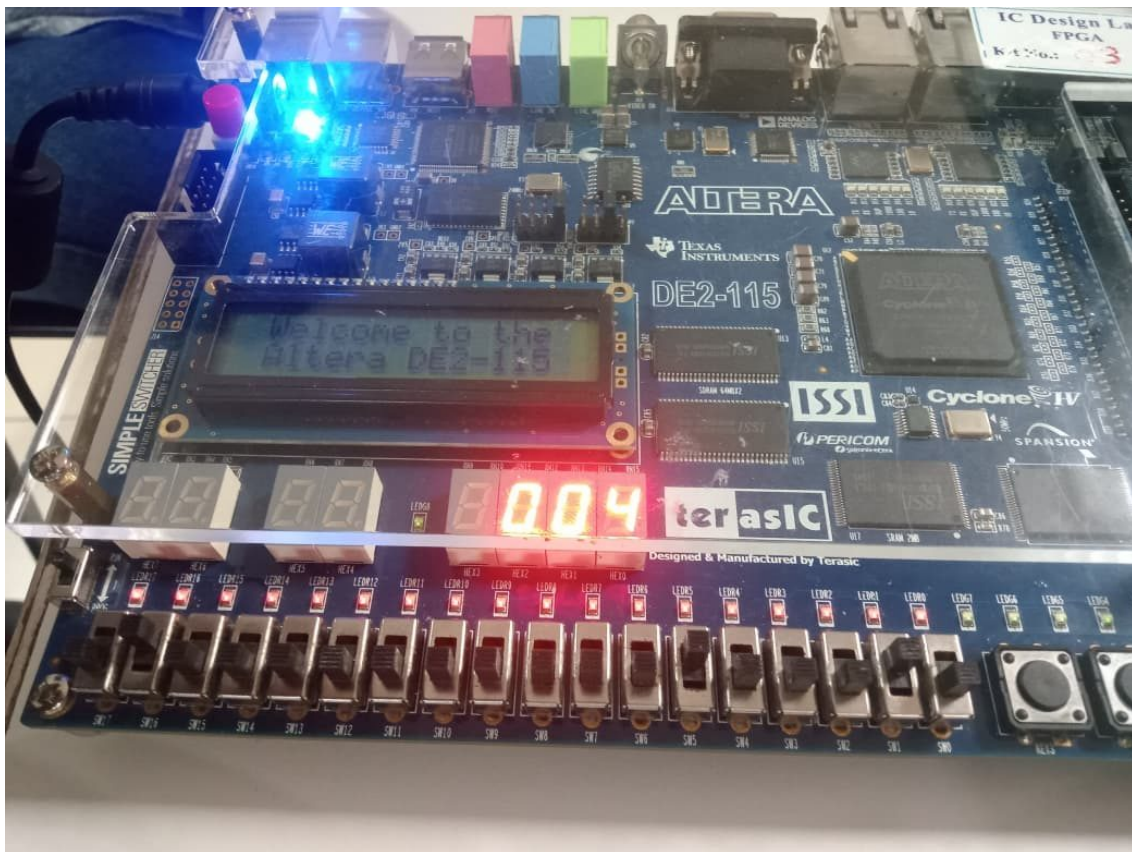


Figure 4: FPGA implementation photograph showing a displayed result of 004.

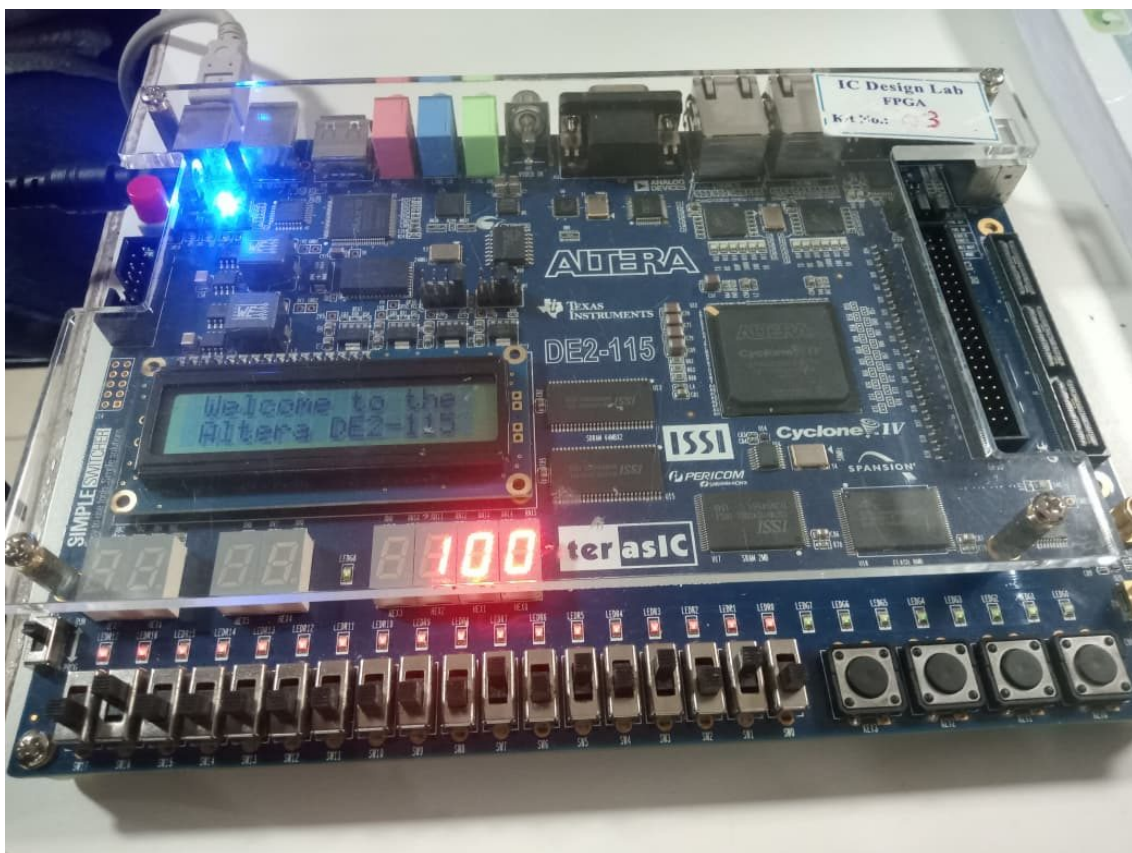


Figure 5: FPGA implementation photograph showing a displayed result of 100.

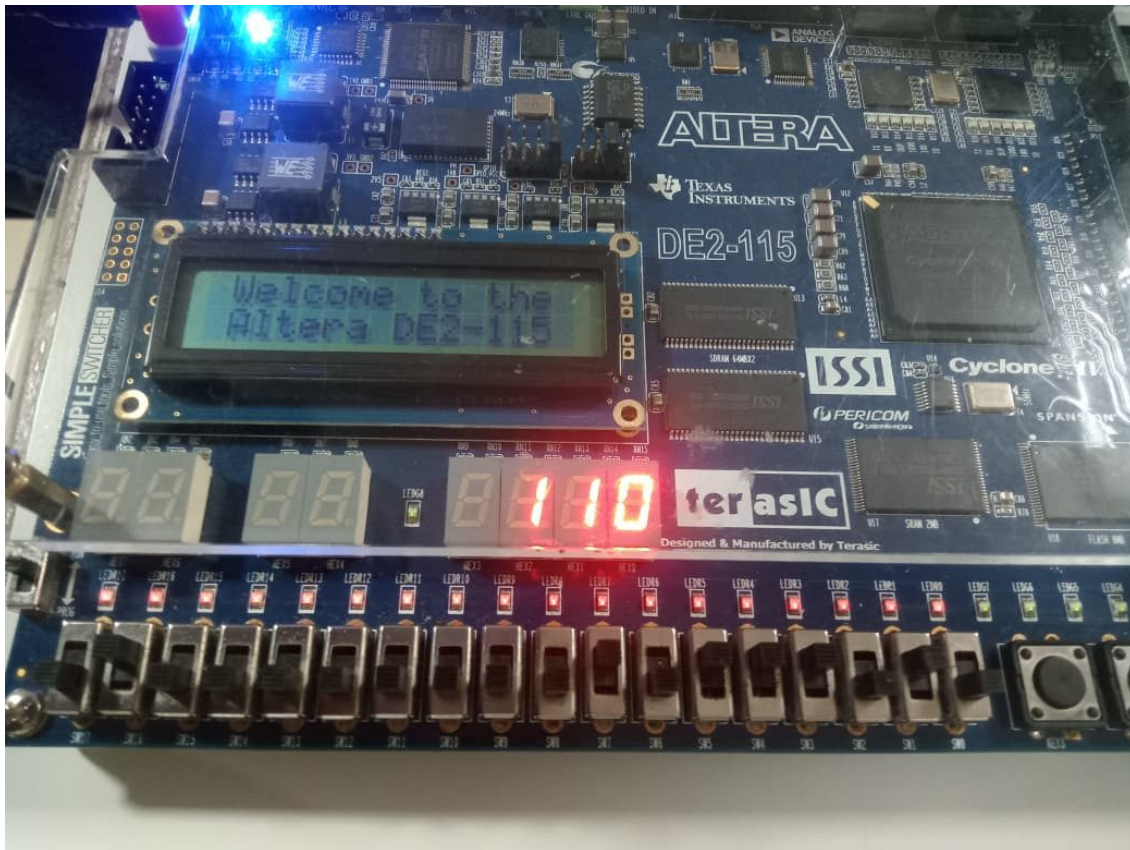


Figure 6: FPGA implementation photograph showing a displayed result of 110.

9. FPGA Project and Pin Assignment

The Quartus project pin assignments used for the DE2-115 FPGA implementation are given below.

```

1 set_global_assignment -name FAMILY "Cyclone IV E"
2 set_global_assignment -name DEVICE EP4CE115F29C7
3 set_global_assignment -name TOP_LEVEL_ENTITY serial_mul
4 set_global_assignment -name VERILOG_FILE adder_4bit.v
5 set_global_assignment -name VERILOG_FILE accumulator_4bit.v
6 set_global_assignment -name VERILOG_FILE datapath_4bit.v
7 set_global_assignment -name VERILOG_FILE control_unit_4bit.v
8 set_global_assignment -name VERILOG_FILE bin_to_bcd.v
9 set_global_assignment -name VERILOG_FILE seven_segment.v
10 set_global_assignment -name VERILOG_FILE serial_mul.v
11
12 # 50 MHz clock
13 set_location_assignment PIN_Y2 -to clk
14
15 # A = multiplicand = SW[3:0]
16 set_location_assignment PIN_AB28 -to multiplicand[0]
17 set_location_assignment PIN_AC28 -to multiplicand[1]
18 set_location_assignment PIN_AC27 -to multiplicand[2]
19 set_location_assignment PIN_AD27 -to multiplicand[3]
20
21 # B = multiplier = SW[7:4]
22 set_location_assignment PIN_AB27 -to multiplier[0]
23 set_location_assignment PIN_AC26 -to multiplier[1]
24 set_location_assignment PIN_AD26 -to multiplier[2]
25 set_location_assignment PIN_AB26 -to multiplier[3]
26
27 # Start and reset switches
28 set_location_assignment PIN_Y24 -to st
29 set_location_assignment PIN_Y23 -to rst
30
31 # HEX0 = ones digit
32 set_location_assignment PIN_G18 -to HEX0[0]
33 set_location_assignment PIN_F22 -to HEX0[1]
34 set_location_assignment PIN_E17 -to HEX0[2]
35 set_location_assignment PIN_L26 -to HEX0[3]

```

```

36 set_location_assignment PIN_L25 -to HEX0[4]
37 set_location_assignment PIN_J22 -to HEX0[5]
38 set_location_assignment PIN_H22 -to HEX0[6]
39
40 # HEX1 = tens digit
41 set_location_assignment PIN_M24 -to HEX1[0]
42 set_location_assignment PIN_Y22 -to HEX1[1]
43 set_location_assignment PIN_W21 -to HEX1[2]
44 set_location_assignment PIN_W22 -to HEX1[3]
45 set_location_assignment PIN_W25 -to HEX1[4]
46 set_location_assignment PIN_U23 -to HEX1[5]
47 set_location_assignment PIN_U24 -to HEX1[6]
48
49 # HEX2 = hundreds digit
50 set_location_assignment PIN_AA25 -to HEX2[0]
51 set_location_assignment PIN_AA26 -to HEX2[1]
52 set_location_assignment PIN_Y25 -to HEX2[2]
53 set_location_assignment PIN_W26 -to HEX2[3]
54 set_location_assignment PIN_Y26 -to HEX2[4]
55 set_location_assignment PIN_W27 -to HEX2[5]
56 set_location_assignment PIN_W28 -to HEX2[6]
57
58 # DONE status on Green LED 0 (not the product output)
59 set_location_assignment PIN_E21 -to done

```

10. Conclusion

The 4-bit serial multiplier was designed and implemented using a finite-state control unit and a datapath containing a 9-bit accumulator and a 4-bit adder. The serial shift/add multiplication algorithm was implemented through the sequence of load, ADD/SKIP and SHIFT operations.

For the test case

$$1010_2 \times 1011_2 = 01101110_2 = 110_{10},$$

the expected product is obtained and displayed on the DE2-115 seven-segment display. The design was verified using ModelSim simulation and FPGA implementation.